

231801034 GENAI 6

```
# STEP 0: Install Required Libraries
!pip install -q faiss-cpu transformers torch pandas

# STEP 1: Load Online Movie Dataset
import pandas as pd

url = "https://raw.githubusercontent.com/sidooms/MovieTweetings/master/
latest/movies.dat"

df = pd.read_csv(url, sep="::", engine="python", header=None) df.columns =
["movieId", "title", "genres"]

print("Dataset loaded successfully")
print(df.head())

# STEP 2: Select Text Column for Embedding
TEXT_COLUMN = "title"
item_texts = df[TEXT_COLUMN].astype(str).tolist()

print("\nTotal movies:", len(item_texts))
print("Sample movie:", item_texts[0])

# STEP 3: Load Transformer Model
from transformers import AutoTokenizer, AutoModel
import torch
import numpy as np
import faiss

device = "cuda" if torch.cuda.is_available() else "cpu"

model_name = "sentence-transformers/all-MiniLM-L6-v2"

tokenizer = AutoTokenizer.from_pretrained(model_name)
encoder = AutoModel.from_pretrained(model_name).to(device) encoder.eval()
```

```

# STEP 4: Encode Text into Embeddings
def encode_texts(texts, batch_size=16):
    all_embeddings = []

    with torch.no_grad():
        for i in range(0, len(texts), batch_size):

            batch = texts[i:i+batch_size]

            tokens = tokenizer(
                batch,
                padding=True,
                truncation=True,
                return_tensors="pt"
            ).to(device)

            outputs = encoder(**tokens)

            embeddings = outputs.last_hidden_state.mean(dim=1)
            embeddings = embeddings.cpu().numpy().astype("float32")

            all_embeddings.append(embeddings)

    return np.vstack(all_embeddings)

item_embeddings = encode_texts(item_texts)

faiss.normalize_L2(item_embeddings)

print("Embeddings shape:", item_embeddings.shape)

# STEP 5: Build FAISS Index
embedding_dim = item_embeddings.shape[1]

index = faiss.IndexFlatIP(embedding_dim)
index.add(item_embeddings)

print("FAISS index built with", index.ntotal, "movies")

# STEP 6: Similarity Search

```

```

def get_similar_items(item_id, k=5):

    query_vector = item_embeddings[item_id:item_id+1]
    scores, indices = index.search(query_vector, k)

    return scores[0], indices[0]

# STEP 7: Test Query
query_id = 0

scores, idxs = get_similar_items(query_id, k=5)

print("\nQUERY MOVIE:")
print(item_texts[query_id])

print("\nSIMILAR MOVIES:")

for score, idx in zip(scores, idxs):
    print(f"-> {item_texts[idx]} (score={score:.3f})")

```